Experiment 01

Student Name: Aryan Yadav **UID: 24BCF10079**
Branch: AIT-CSE (Full Stack) **Section/Group: 24AIT_KRG-G2**
Semester: 5

Aim

To study and implement advanced SQL queries using conditional logic, aggregate functions, joins, subqueries, self-joins, and date operations to solve real-world database problems.

Question 1

Write an SQL solution to calculate the total number of bank accounts belonging to each salary category.

The salary categories are:

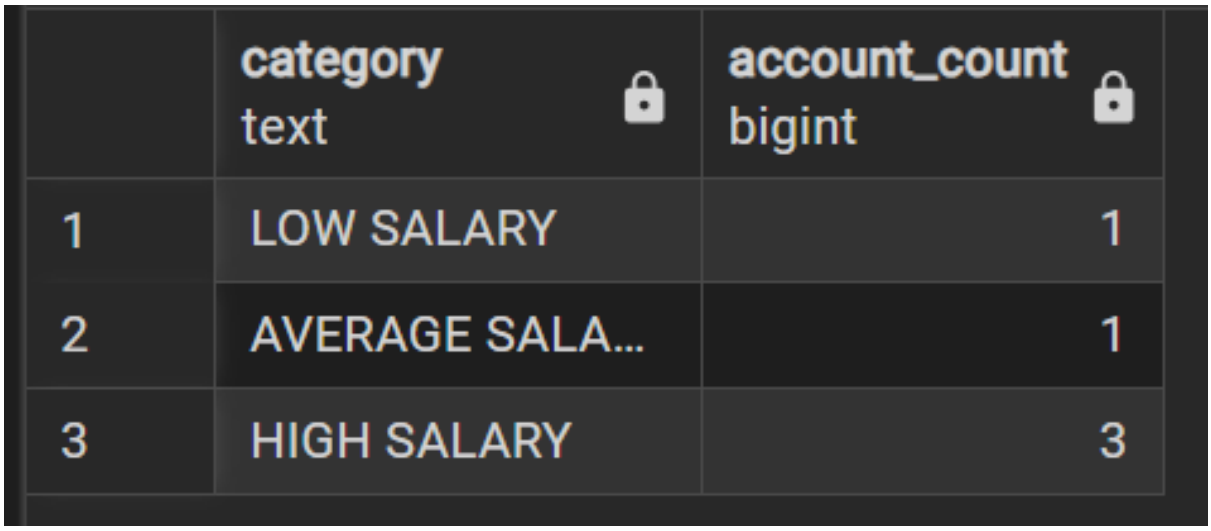
- Low Salary: Monthly salary less than 20000
- Average Salary: Monthly salary between 20000 and 50000 (inclusive)
- High Salary: Monthly salary greater than 50000

The final output should always contain all three salary categories even if one category contains zero accounts.

Answer

```sql
SELECT 'LOW SALARY' AS CATEGORY, COUNT(ACCOUNT_ID) AS ACCOUNT_COUNT
    FROM ACCOUNTS
    WHERE INCOME < 20000

    UNION ALL

SELECT 'AVERAGE SALARY' AS CATEGORY, COUNT(ACCOUNT_ID) AS
   ACCOUNT_COUNT
    FROM ACCOUNTS
    WHERE INCOME >= 20000 AND INCOME <= 50000

    UNION ALL

SELECT 'HIGH SALARY' AS CATEGORY, COUNT(ACCOUNT_ID) AS ACCOUNT_COUNT
    FROM ACCOUNTS
    WHERE INCOME >50000;
```

Output



	category text	account_count bigint
1	LOW SALARY	1
2	AVERAGE SALA...	1
3	HIGH SALARY	3

Question 2

Design an Employee Management System database architecture by performing the following operations:

- Create Departments, Employees and Salaries tables.
- Insert sample records.
- Display employee details using JOIN.
- Increase HR employee salary by 10%.
- Delete salary records below 30000.
- Display employees earning more than the company average salary.

Answer

Create Department Table

```sql
CREATE TABLE Departments
(
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(50)
);
```

Create Employee Table

```sql
CREATE TABLE Employees
(
    emp_id INT PRIMARY KEY,
    name VARCHAR(50),
    dept_id INT,
    FOREIGN KEY(dept_id)
    REFERENCES Departments(dept_id)
);
```

Create Salary Table

```sql
CREATE TABLE Salaries
(
    emp_id INT,
    salary NUMERIC(10,2),
    FOREIGN KEY(emp_id)
    REFERENCES Employees(emp_id)
);
```

Insert Sample Data

```sql
INSERT INTO Departments VALUES
(1,'HR'),
(2,'IT'),
(3,'Finance');

INSERT INTO Employees VALUES
(101,'Alice',1),
(102,'Bob',2),
(103,'Charlie',1),
(104,'David',3);

INSERT INTO Salaries VALUES
(101,35000),
(102,60000),
(103,28000),
(104,45000);
```

Display Employee Details

```sql
SELECT
e.name,
d.dept_name,
s.salary

FROM Employees e

JOIN Departments d
ON e.dept_id=d.dept_id

JOIN Salaries s
ON e.emp_id=s.emp_id;
```

Increase HR Salary by 10%

```sql
UPDATE Salaries

SET salary=salary*1.10

WHERE emp_id IN
(
SELECT emp_id
FROM Employees
```

```sql
WHERE dept_id=
(
SELECT dept_id
FROM Departments
WHERE dept_name='HR'
)
);
```

Delete Salary Below 30000

```sql
DELETE FROM Salaries
WHERE salary<30000;
```

Employees Above Company Average Salary

```sql
SELECT
e.name,
s.salary

FROM Employees e

JOIN Salaries s
ON e.emp_id=s.emp_id

WHERE salary>
(
SELECT AVG(salary)
FROM Salaries
);
```

Output

	name character varying (50)	dept_name character varying (50)	salary integer
1	Alice	HR	50000
2	Bob	HR	25000
3	Charlie	IT	80000
4	David	IT	40000
5	Emma	Sales	45000

Output

	name character varying (50)	dept_name character varying (50)	salary integer
1	Charlie	IT	80000
2	David	IT	40000
3	Emma	Sales	45000
4	Alice	HR	55000
5	Bob	HR	27500

Output

	name character varying (50)	salary integer
1	Charlie	80000
2	Alice	55000

Question 3

Generate every possible pizza consisting of exactly three different toppings. Display the topping names separated by commas along with the total ingredient cost. Sort the result by highest cost and alphabetically for ties.

Answer

```
SELECT

CONCAT_WS(', ',
t1.topping_name,
t2.topping_name,
t3.topping_name) AS pizza,

(t1.ingredient_cost+
t2.ingredient_cost+
```

```
10 t3.ingredient_cost) AS total_cost
11
12 FROM pizza_toppings t1
13
14 JOIN pizza_toppings t2
15 ON t1.topping_name<t2.topping_name
16
17 JOIN pizza_toppings t3
18 ON t2.topping_name<t3.topping_name
19
20 ORDER BY total_cost DESC,
21 pizza;
```

Output

	pizza text	total_cost numeric
1	Chicken, Pepperoni, Sausage	1.75
2	Chicken, Extra Cheese, Sausage	1.65
3	Extra Cheese, Pepperoni, Saus...	1.60
4	Chicken, Onions, Sausage	1.50
5	Chicken, Extra Cheese, Pepper...	1.45
6	Onions, Pepperoni, Sausage	1.45
7	Extra Cheese, Onions, Sausage	1.35
8	Chicken, Onions, Pepperoni	1.30
9	Chicken, Extra Cheese, Onions	1.20
10	Extra Cheese, Onions, Pepperoni	1.15

Question 4

Write a PostgreSQL query to identify users who made another purchase within 1 to 7 days of an earlier purchase. Display only the unique User IDs sorted in ascending order.

Answer

```sql
SELECT DISTINCT

t1.user_id

FROM amazon_transactions t1

JOIN amazon_transactions t2

ON t1.user_id=t2.user_id

AND t2.created_at>t1.created_at

AND t2.created_at<=
t1.created_at+INTERVAL '7 days'

ORDER BY user_id;
```

Output

	user_id integer
1	101
2	103

Learning Outcome

After completing this experiment, the following learning outcomes were achieved.

- Understood advanced SQL query formulation.
- Applied conditional logic using CASE statements.
- Implemented aggregate functions and GROUP BY operations.
- Performed table creation, insertion, update and deletion operations.
- Applied INNER JOIN and SELF JOIN for relational data retrieval.

- Used subqueries to perform complex filtering and updates.
- Generated unique combinations using self joins.
- Performed date arithmetic for transaction analysis.
- Improved practical knowledge of PostgreSQL and relational database management.